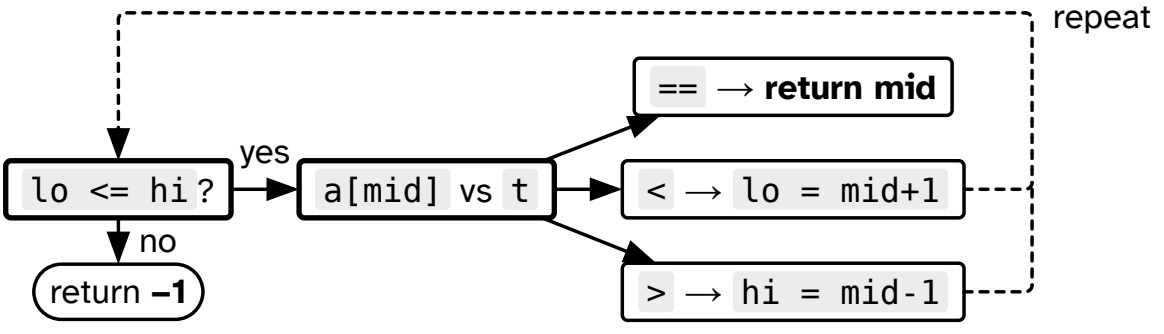
Binary Search

Template

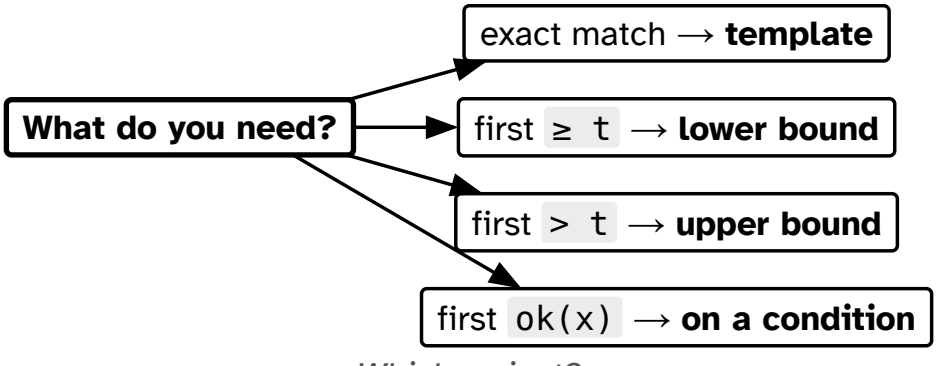
```
lo, hi = 0, len(a) - 1
while lo <= hi:
    mid = lo + (hi - lo) // 2
    if a[mid] == t: return mid
    if a[mid] < t: lo = mid + 1
    else:          hi = mid - 1
return -1
```



One loop iteration

Variants

Goal	Change
First ≥ t (lower bound)	`while lo < hi`, `hi = len(a)`; if `a[mid] < t` then `lo = mid+1` else `hi = mid`; return `lo`
First > t (upper bound)	same, but test `a[mid] <= t`
On a condition	find the first `x` where `ok(x)` is true; `ok` must be monotonic

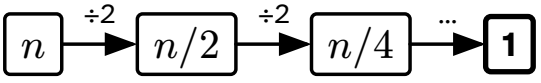


Which variant?

Complexity

$$T(n) = T\left(\frac{n}{2}\right) + O(1) = O(\log n)$$

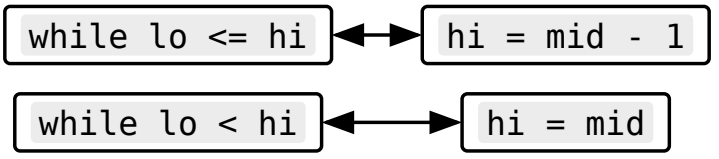
- Iterative: $O(1)$ space · Recursive: $O(\log n)$ space
- $n = 10^6 \longrightarrow$ at most 20 steps · $n = 10^9 \longrightarrow$ at most 30 steps



Each step halves the search space: about $\log_2 n$ steps

Traps

- Overflow: use `lo + (hi-lo)//2`
- `lo < hi` goes with `hi = mid`; `lo <= hi` goes with `hi = mid - 1`
- Unsorted input → wrong answers, but no error



Match the loop condition to the update